

Manual del Programador - Versión 1.0.0

Proyecto: Sistema / TMC (Proyecto Integrador Anual)

Integrantes: Del Pino, Mattia, Orue, Ferrari

Institución: EEST N.º 1 “Eduardo Ader” - Vicente López

Materia: Laboratorio de Programación (LPR)

1. Introducción y Propósito

Este manual técnico documenta la arquitectura de software y el uso de estructuras de datos (`struct`) combinadas con punteros en C++, implementadas para el modelado de entidades en el prototipo del proyecto integrador. Su propósito es servir de guía de referencia para los integrantes del equipo de desarrollo en el manejo eficiente de la memoria RAM y el flujo de entrada de datos

2. Arquitectura del Código: Estructuras y Punteros

2.1. Definición del `struct`

Para representar entidades complejas (como módulos de hardware, sensores o registros de usuario), se utiliza una estructura homogénea que agrupa variables de diferentes tipos de datos en bloques contiguos de memoria[cite: 1]:

```
```cpp
struct EntidadProyecto {
 int id; // Identificador único
 char nombre[50]; // Descripción o nombre textual
 float metrica; // Valor flotante para lecturas o métricas
};
```

#### 2.2. Pasaje por Dirección y el Operador Flecha (->)

En lugar de pasar objetos por valor (lo cual genera copias pesadas innecesarias en el Stack), las funciones reciben la dirección de memoria física del objeto utilizando un puntero (\*) y el operador de indirección o flecha (->) para manipular los datos originales directamente[cite: 1]:

```
C++
void cargarDatos(EntidadProyecto* ptr) {
 cout << "=> Ingrese el ID de la entidad: ";
 cin >> ptr->id;

 cin.ignore(); // Limpieza obligatoria del búfer
```

```
cout << "=> Ingrese el Nombre: ";
cin.getline(ptr->nombre, 50);

cout << "=> Ingrese la Metrica: ";
cin >> ptr->metrica;
}
```

### 3. Manejo del Búfer de Entrada (**cin.ignore**)

Un aspecto crítico al combinar lecturas numéricas (**cin >>**) con cadenas de texto (**cin.getline**) es la gestión del búfer de entrada.

- **El problema:** Al presionar *Enter* tras ingresar un número, queda un salto de línea residual (**\n**) almacenado temporalmente en el búfer.
- **La solución:** La inclusión de **cin.ignore()** justo antes de capturar cadenas de texto elimina dicho carácter remanente, evitando que el programa se salte la lectura de **cin.getline** por error[cite: 1].

### 4. Guía de Compilación y Ejecución Local

Para compilar y verificar el funcionamiento del módulo desde la terminal de Visual Studio Code (PowerShell) utilizando el entorno GNU C++[cite: 1]:

PowerShell

```
Compilación del código fuente generando el ejecutable
g++ src/main.cpp -o src/sistema.exe
```

```
Ejecución de la aplicación
.\src\sistema.exe
```